Atos Worldline NV/SA
Haachtsesteenweg 1442 Chaussée de Haecht
B-1130 Brussels
Belgium
http://www.atosworldline.be

TITLE Electronic Cash Register Integration - Implementation Guide

VIC 1.07/13

AUTHOR(S) Rudy Sterckx, Damien Miliche

STATUS FINAL

CONFIDENTIALITY NOT APPLICABLE

ACCEPTED BY

DISTRIBUTION LIST Atos Worldline, ECR-manufacturers.

DOCUMENT SUMMARY

VERSION / RELEASE MANAGEMENT REPORT

Version	Name(s)	Date	Comments
1.07/11 **1.0**	Bart Warreyn	04/12/06	new version
1.07/11 **3.0**	Bart Warreyn	02/07/07	- p.13: New flowchart – new behaviour for iep_tx_inc = 5015. - p.19: changed “or” into “and” in conditions for error display. - p. 26: TINA back-up: Format of file_identification in record 00 is 64a.
1.07/11 **4.0**	Rudy Sterckx	10/06/10	- p.14: New flowchart – Treatment of result of the transaction by VMC. - Remove of ATEA transaction
1.07/11 **4.1**	Rudy Sterckx	08/02/11	- p.19: Incident 4009: Consider this incident “uncertain”.
1.07/13 **5.0**	Damien Miliche	06/01/12	Update for VIC 1.07/13
1.07/13 **5.1**	Damien Miliche	08/06/12	Explicit incident code 00 01 for partial approval in PD message
1.07/13 **5.2**	Damien Miliche	31/08/12	correct length of field vic_bit_map in Section 4.4 and 4.5 correct length of field term_id (8 bytes instead of 4 for a 8a field)
1.07/13 **5.3**	Damien Miliche	19/10/12	correct format of field pdv_state (32b instead of 8x)
1.07/13 **5.4**	Damien Miliche	13/02/13	correct vic_bit_map_application_id bit for BC/MC EMV (bit 66 instead of 65)

TABLE OF CONTENTS

1. INTRODUCTION 5

1.1 Purpose of this document 5

1.2 Referenced documents 5

1.3 Main differences with previous versions of the protocol 5

1.4 Reading convention 7

1.5 RELEVANT PROTOCOL LAYERS 7

2. KISS LAYER 8

2.1 Sending a frame: 8

2.2 Receiving a frame: 12

3. VIC-LAYER - INTRODUCTION 13

3.1 Contents and layout of VIC-messages in general 13

3.2 Messages needed to perform a transaction 13

4. PERFORMING TRANSACTIONS 15

4.1 Supported transaction types 15
4.1.1 BC/MC R3bc 15
4.1.2 Proton 15
4.1.3 EMV (including BC/MC EMV) 15

4.2 vmc_debit_request 16

4.3 pdv_debit_result 17
4.3.1 Layout *pdv_debit_result* of a transaction performed by the Proprietary Application: 19
4.3.2 Layout *pdv_debit_result* of a transaction performed by the Common Application: 20

4.4 pdv_error 22

4.5 vmc_cancel 22
4.5.1 vmc_cancel at start-up 22
4.5.2 vmc_cancel for transaction recovery 22

4.6 Error codes - translations 24

5. OTHER FUNCTIONALITIES 26

5.1 Special credit card operations 26
5.1.1 Sale cancel (00) 26
5.1.2 Mail order sale 27

5.2 Proton configuration 28

5.3 TINA 30
5.3.1 TINA Activation 30
5.3.2 TINA Transaction 31
5.3.3 TINA Deactivation 31
5.3.4 TINA Consultation 32
5.3.5 TINA Back-up 33

5.4 TINA EMV 36
5.4.1 TINA EMV Activation 36
5.4.2 TINA EMV Transaction 37
5.4.3 TINA EMV Deactivation 39
5.4.4 TINA EMV Consultation 40
5.4.5 TINA EMV Recovery 41

5.5 Partial approval 42

5.6 Tokenization 43

5.7 Counters consultation and counters closing 44

5.8 Belgian ID card reading 46

5.9 Belgian SIS card reading (Social Identity System) 47

1. INTRODUCTION

1.1 Purpose of this document

This document describes how external devices can use the VIC-protocol to communicate with an Atos Worldline payment terminal in order to perform transactions in a correct and safe way.
It is assumed:

- that the payment terminal works in the 'NEW VIC'-mode (referred to as NVIC in the terminal menu);
- that no other currency than EURO is used.

1.2 Referenced documents

This document is based on (**and is not intended as a replacement of**) the more detailed document:
"***Reference Functional Specifications, VIC Standard"***
(version 1.07/13)

[text_ref] List of text references
[recovery_file] Atos Worldline – Application Note: Recovery File (Feb 2012)

1.3 Main differences with previous versions of the protocol

vic_version

The field ***vic_version*** which was present only in the messages **vmc_purse_request**, **pdv_purse_result**, **vmc_debit_request** and **pdv_debit_result** is now present in all messages.

card_id_disp

Since it has become useless, the field ***card_id_disp*** has been removed from the message **vmc_debit_request**.

ticket_data/display_data

From this version on, the field ***ticket_data*** of the message **pdv_debit_result** will contain only the data which needs to be printed. Control characters 0x11 (DC1 – printer select) and 0x13 (DC3 – printer deselect) are no longer used.

Consequently, a new field ***display_data*** is introduced which will contain only the data which needs to be shown on the operator display.

vic_proprietary_data

A new field ***vic_proprietary_data*** is introduced in the message ***vmc_debit_request*** and ***pdv_debit_result***. This field is used to transport specific proprietary data.

acquirer_id

A new field ***acquirer_id*** is introduced in the message **pdv_debit_result**, it identifies the acquirer through which the payment was done. Useful in situations where multiple acquirers process the same card brand. This field is also used in the ***vmc_command*** and ***pdv_command_reply*** for the closing or consultation of counters.

ATEA

Transactions processed using ATEA-protocol between payment terminal and acquirer host are no longer supported.

term_id

The format of this field has been adapted from 8i to 8a.

display_text_ref

Having the same function as the ***display_data*** field, the ***display_text_ref*** enables the terminal to inform the ECR to display some message. However, as opposed to the ***display_data*** field, enabling to transmit free text, the ***display_text_ref*** enables to transmit a text reference, corresponding to a specified message described in [text_ref].

pdv_state/pdv_state_mask

A new information is added in this fields, enabling the ECR to activate and receive synchronization messages for updating of its display.

1.4 Reading convention

All values specified in the description of the protocol-messages are hexadecimal;
each value is 1 byte ; bytes are always separated by a blank.

Example **12 41 EF 00**
Means 4 bytes (8 nibbles) of data
*(the second byte would be '**A**' in ASCII-format, but this document uses only hexadecimal)*

Abbreviations used :
VMC = External device (ECR, PC with point-of-sale software...)
PDV = Atos Worldline device (XENTA, YOMANI…)
BC/MC = Bancontact/Mister Cash

1.5 RELEVANT PROTOCOL LAYERS

Both VMC and PDV should consider two different communication protocols (layers) and treat them separately **(as does this document)**:

- The VIC protocol is the upper protocol layer used for the communication between the application of the external device (VMC) and the application of the payment terminal (PDV)
- All VIC-messages are transported using the lower protocol layer called KISS. This telecommunication protocol assures that VIC-messages are transported in a reliable, unambiguous way.

Note on the hardware layer:
Serial communication over RS232.
Port settings: 9600bps, no parity, 8 data bits, 1 stop bit, no flow control

2. KISS layer

2.1 Sending a frame:

VIC-messages should be sent in a KISS-frame as shown below :

02 (STX)	[**00..FF**] (SEQ)	**VIC-message** (containing DLE if necessary)	**03** (ETX)	[**00..FF**] CRC(lsb)	[**00..FF**] CRC(msb)

STX
Marks the beginning of a frame and is always **02**.

SEQ
= counter: every time a device sends a frame, this number should be incremented.

*At start-up or after desynchronisation, SEQ of the first frame sent must have value **FF**. Next frame should have SEQ **00**, next frame SEQ **01** and so on. If the value **FE** is reached, the next frame may not be sent with SEQ **FF**, the counter must be reset to **00.***

ETX
Marks the end of a frame and is always **03**.

DLE
Is used for byte stuffing and is always **10**. This mechanism can be compared to the '\'-escape sequence used in C(++).

If the VIC-message contains data bytes **02** and/or **03**, the KISS layer must add a DLE-byte just before these data bytes, in order to distinguish them from STX or ETX. Consequently, if the VIC-message contains a data byte **10**, the KISS layer must also add a DLE just before.

Example:
If VIC-layer wants to send the following message:
01 02 03 04 CD 10 10 11 12

the KISS-layer will, before adding the STX, SEQ, ETX and CRC's, convert this to :
01 10 02 10 03 04 CD 10 10 10 10 11 12

CRC (lsb) & CRC (msb)
In order to enable the receiver to check if the frame has been transported without errors, these two check bytes must be attached after the frame. These check bytes are the result of an algorithm applied on all the bytes of the frame except STX (including DLE bytes).

Before applying the algorithm, both CRC (lsb) and CRC (msb) are equal to **00**.
The algorithm applied on all data bytes of the frame is:

table_index = CRC(lsb) XOR data_byte
CRC(lsb) = CRC(msb) XOR lsb(table(table_index))
CRC(msb) = 00 XOR msb(table(table_index))

***lsb/msb** = least significant byte/most significant byte.*

The table used by the algorithm can be found on the next page.
(exists also in *.csv-format)

Example : CRC-computation on the frame 02 05 AB 03 (AB being the VIC-message)

initial CRC(lsb)= 00
initial CRC(msb)= 00

1st byte = 05		
table_index = CRC(lsb) XOR data_byte	**crc(lsb)** = crc(msb) XOR lsb(table(table_index))	**crc(msb)** = 00 XOR msb(table(table_index))
table_index = 00 XOR 05	crc(lsb) = 00 XOR C0	crc(msb) = 00 XOR 03
table_index = 0000 0000 XOR 0000 0101	crc(lsb) = 0000 0000 XOR 1100 0000	crc(msb) = 0000 0000 XOR 0000 0011
table_index = 0000 0101	crc(lsb) = 1100 0000	crc(msb) = 0000 0011
table_index = **05**	crc(lsb) = **C0**	crc(msb) = **03**
		CRC(lsb) = C0 – CRC(msb) = 03

2nd byte = AB		
table_index = CRC(lsb) XOR data_byte	**crc(lsb)** = crc(msb) XOR lsb(table(table_index))	**crc(msb)** = 00 XOR msb(table(table_index))
table_index = C0 XOR AB	crc(lsb) = 03 XOR 41	crc(msb) = 00 XOR EF
table_index = 1100 0000 XOR 1010 1011	crc(lsb) = 0000 0011 XOR 0100 0001	crc(msb) = 0000 0000 XOR 1110 1111
table_index = 0110 1011	crc(lsb) = 0100 0010	crc(msb) = 1110 1111
table_index = **6B**	crc(lsb) = **42**	crc(msb) = **EF**
		CRC(lsb) = 42 – CRC(msb) = EF

3rd byte = 03		
table_index = CRC(lsb) XOR data_byte	**crc(lsb)** = crc(msb) XOR lsb(table(table_index))	**crc(msb)** = 00 XOR msb(table(table_index))
table_index = 42 XOR 03	crc(lsb) = EF XOR C0	crc(msb) = 00 XOR 30
table_index = 0100 0010 XOR 0000 0011	crc(lsb) = 1110 1111 XOR 1100 0000	crc(msb) = 0000 0000 XOR 0011 0000
table_index = 0100 0001	crc(lsb) = 0010 1111	crc(msb) = 0011 0000
table_index = **41**	crc(lsb) = **2F**	crc(msb) = **30**
		CRC(lsb) = 2F – CRC(msb) = 30

The frame must be sent as follows : 02 05 AB 03 **2F 30**

table index	value	table index	value	table index	value	table index	value	
00	0x0000	40	0xf001	80	0xa001	C0	0x5000	
01	0xc0c1	41	0x30c0	81	0x60c0	C1	0x90c1	
02	0xc181	42	0x3180	82	0x6180	C2	0x9181	
03	0x0140	43	0xf141	83	0xa141	C3	0x5140	
04	0xc301	44	0x3300	84	0x6300	C4	0x9301	
05	0x03c0	45	0xf3c1	85	0xa3c1	C5	0x53c0	
06	0x0280	46	0xf281	86	0xa281	C6	0x5280	
07	0xc241	47	0x3240	87	0x6240	C7	0x9241	
08	0xc601	48	0x3600	88	0x6600	C8	0x9601	
09	0x06c0	49	0xf6c1	89	0xa6c1	C9	0x56c0	
0A	0x0780	4A	0xf781	8A	0xa781	CA	0x5780	
0B	0xc741	4B	0x3740	8B	0x6740	CB	0x9741	
0C	0x0500	4C	0xf501	8C	0xa501	CC	0x5500	
0D	0xc5c1	4D	0x35c0	8D	0x65c0	CD	0x95c1	
0E	0xc481	4E	0x3480	8E	0x6480	CE	0x9481	
0F	0x0440	4F	0xf441	8F	0xa441	CF	0x5440	
10	0xcc01	50	0x3c00	90	0x6c00	D0	0x9c01	
11	0x0cc0	51	0xfcc1	91	0xacc1	D1	0x5cc0	
12	0x0d80	52	0xfd81	92	0xad81	D2	0x5d80	
13	0xcd41	53	0x3d40	93	0x6d40	D3	0x9d41	
14	0x0f00	54	0xff01	94	0xaf01	D4	0x5f00	
15	0xcfc1	55	0x3fc0	95	0x6fc0	D5	0x9fc1	
16	0xce81	56	0x3e80	96	0x6e80	D6	0x9e81	
17	0x0e40	57	0xfe41	97	0xae41	D7	0x5e40	
18	0x0a00	58	0xfa01	98	0xaa01	D8	0x5a00	
19	0xcac1	59	0x3ac0	99	0x6ac0	D9	0x9ac1	
1A	0xcb81	5A	0x3b80	9A	0x6b80	DA	0x9b81	
1B	0x0b40	5B	0xfb41	9B	0xab41	DB	0x5b40	
1C	0xc901	5C	0x3900	9C	0x6900	DC	0x9901	
1D	0x09c0	5D	0xf9c1	9D	0xa9c1	DD	0x59c0	
1E	0x0880	5E	0xf881	9E	0xa881	DE	0x5880	
1F	0xc841	5F	0x3840	9F	0x6840	DF	0x9841	
20	0xd801	60	0x2800	A0	0x7800	E0	0x8801	
21	0x18c0	61	0xe8c1	A1	0xb8c1	E1	0x48c0	
22	0x1980	62	0xe981	A2	0xb981	E2	0x4980	
23	0xd941	63	0x2940	A3	0x7940	E3	0x8941	
24	0x1b00	64	0xeb01	A4	0xbb01	E4	0x4b00	
25	0xdbc1	65	0x2bc0	A5	0x7bc0	E5	0x8bc1	
26	0xda81	66	0x2a80	A6	0x7a80	E6	0x8a81	
27	0x1a40	67	0xea41	A7	0xba41	E7	0x4a40	
28	0x1e00	68	0xee01	A8	0xbe01	E8	0x4e00	
29	0xdec1	69	0x2ec0	A9	0x7ec0	E9	0x8ec1	
2A	0xdf81	6A	0x2f80	AA	0x7f80	EA	0x8f81	
2B	0x1f40	6B	0xef41	AB	0xbf41	EB	0x4f40	
2C	0xdd01	6C	0x2d00	AC	0x7d00	EC	0x8d01	
2D	0x1dc0	6D	0xedc1	AD	0xbdc1	ED	0x4dc0	
2E	0x1c80	6E	0xec81	AE	0xbc81	EE	0x4c80	
2F	0xdc41	6F	0x2c40	AF	0x7c40	EF	0x8c41	
30	0x1400	70	0xe401	B0	0xb401	F0	0x4400	
31	0xd4c1	71	0x24c0	B1	0x74c0	F1	0x84c1	
32	0xd581	72	0x2580	B2	0x7580	F2	0x8581	
33	0x1540	73	0xe541	B3	0xb541	F3	0x4540	
34	0xd701	74	0x2700	B4	0x7700	F4	0x8701	
35	0x17c0	75	0xe7c1	B5	0xb7c1	F5	0x47c0	
36	0x1680	76	0xe681	B6	0xb681	F6	0x4680	
37	0xd641	77	0x2640	B7	0x7640	F7	0x8641	
38	0xd201	78	0x2200	B8	0x7200	F8	0x8201	
39	0x12c0	79	0xe2c1	B9	0xb2c1	F9	0x42c0	
3A	0x1380	7A	0xe381	BA	0xb381	FA	0x4380	
3B	0xd341	7B	0x2340	BB	0x7340	FB	0x8341	
3C	0x1100	7C	0xe101	BC	0xb101	FC	0x4100	
3D	0xd1c1	7D	0x21c0	BD	0x71c0	FD	0x81c1	
3E	0xd081	7E	0x2080	BE	0x7080	FE	0x8081	
3F	0x1040	7F	0xe041	BF	0xb041	FF	0x4040	

After having sent the frame, the sender should calculate the answer time-out:

Answer time-out (in sec) = ((frame length in bytes*10)/9600) * 2

(*example : for a frame length of 100 bytes, the answer time-out is 0,208 sec = 208 milliseconds*)

The sender should wait for an answer from the receiver during this time-out.

To avoid calculating this answer time-out on all frames that are sent, it is acceptable to use a fixed time-out of 900 milliseconds for every frame.

Possible answers of the receiver and expected actions from the sender:

- *In the exceptional case of the receiver answering with "Wait for Acknowledgement" WAK (**13**), the answer time-out should be reset to zero and the sender must continue waiting for another reaction of the receiver.*

- The sender should resend the frame (with a maximum of three retries) if the receiver :
  - does not answer before expiry of the answer time-out
  - answers with a negative acknowledgement NAK (**15**) or any other incoherent data

- No further action of the sender is required if a positive acknowledgement was sent by the receiver. The frame is sent successfully

  Layout of a positive acknowledgement :

06 (ACK)	[**00..FF**] (SEQ)

(*SEQ should be equal to the sequence number used in the sent frame*)

2.2 Receiving a frame:

The KISS-layer of the VMC must be able to receive frames at all times, regardless of the state of the VIC-layer.

VIC-messages are received in a KISS-frame as shown below:

02 (STX)	[**00..FF**] (SEQ)	**VIC-message** (containing DLE if necessary)	**03** (ETX)	[**00..FF**] CRC(lsb)	[**00..FF**] CRC(msb)

(Description of STX, ETX etc…, cfr. supra: "*sending a VIC-message"*)

Preliminary rules:

- A frame received must always begin with STX. If data bytes not preceded by STX are received, they should simply be ignored.
- A frame must be considered as ended :
  - when the bytes ETX CRC(lsb) CRC(msb) are received.
  - when no byte is received during more than 200 milliseconds (inter-character time-out)

Actions to be undertaken by the sender after receiving a frame:

- The receiver must perform a CRC-computation over the data bytes received and check if the same values are found.

- The receiver must reply to the sender with :

positive acknowledgment (ACK)

06 (ACK)	[**00..FF**] (SEQ)

or

negative acknowledgment (NAK)

15 (NAK)

This answer must be prompted: cfr. higher "*answer time-out"*

By sending NAK, the receiver informs the sender that the transmission of the frame was unsuccessful.
By sending ACK SEQ, the receiver informs the sender that the transmission of the frame was successful.

*Conditions for sending **NAK**:*

- an inter-character timeout occurred.
- the CRC check failed, i.e. the values found by the receiver are different from the ones sent.

*Condition for sending **ACK SEQ**:*

- the CRC check succeeded, i.e. the values found by the receiver are equal to the ones sent.

Before transferring the received message to the VIC-layer, the KISS-bytes should be removed.
(STX, SEQ, ETX and CRC's as well as all DLE's present in the VIC-part)

3. VIC-LAYER - INTRODUCTION

3.1 Contents and layout of VIC-messages in general

Layout of a VIC-message:

vic_protocol_id (2 bytes)	***vic_msg_code*** (2 bytes)	***vic_bitmap*** (8,16 or 24 bytes)	field 1	field 2	… …	field n

- The field ***vic_protocol_id*** identifies the version of the protocol used. Since we are currently using VIC 1.07, it will always contain **01 07**. If the PDV replies with messages containing a value different from **01 07**, there is a version incompatibility and the transaction processing cannot be reliable.

- The field ***vic_msg_code*** identifies the message.
  Example : **56 44** (VD) if the message is a **vmc_debit_request**.

- The field ***vic_bitmap*** indicates which fields will follow in the message.

  All fields of the VIC-protocol (except ***vic_protocol_id*** and ***vic_msg_code***) carry a unique index.
  In the field ***vic_bitmap***, the bits corresponding to the indexes of the field present will be set.

  Example (meaningless in VIC) :
  If fields 5,9,15,34 and 64 are present in a message, the bits of its ***vic_bitmap*** will be set as follows :

  0000 1000 1000 0010 0000 0000 0000 0000 0100 0000 0000 0000 0000 0000 0000 0001

  After conversion to bytes, the ***vic_bitmap*** to be sent will be :

  08 82 00 00 40 00 00 01

  Since the unique indexes of the fields of the VIC protocol have serial numbers between 1 and 192, the bitmap needs to be extended with 8 bytes if the message contains fields with serial numbers between 65 and 128 and a further extension with 8 bytes if fields with serial numbers higher than 128 are present.
  The first bit of every series of 8 bytes in the bitmap (ie. **bit 1** and **bit 65**) indicates the presence of a vic_bitmap extension: if it is set, 8 more bytes will follow.

VERY IMPORTANT:

Upon receiving a message, it is important to analyse the field ***vic_bitmap*** to find out which fields are present in the message before parsing it. Contents of messages may change at any time.

3.2 Messages needed to perform a transaction

The four messages of the minimum protocol (the messages needed to perform transactions) are:

- messages sent by the VMC
  - **vmc_debit_request** ***vic_msg_code*** = **56 44** (VD)
  - **vmc_cancel** ***vic_msg_code*** = **56 43** (VC)

- messages sent by the PDV
  - **pdv_debit_result** ***vic_msg_code*** = **50 44** (PD)
  - **pdv_error** ***vic_msg_code*** = **50 45** (PE)

Other messages are defined in the VIC specifications; their use is aimed at specific functionalities as described in Section 5 and are not necessary to perform a simple transaction:

- optional messages sent by the VMC
  - **vmc_purse_request** ***vic_msg_code*** = **56 50** (VP)
  - **vmc_command** ***vic_msg_code*** = **56 4D** (VM)
  - **vmc_data** ***vic_msg_code*** **= 56 4F** (VO)

- optional messages sent by the PDV
  - **pdv_purse_result** ***vic_msg_code*** = **50 50** (PP)
  - **pdv_command_reply** ***vic_msg_code*** = **50 4D** (PM)
  - **pdv_data** ***vic_msg_code*** **= 56 4F** (VD)

4. PERFORMING TRANSACTIONS

4.1 Supported transaction types

Atos Worldline PDVs support different types of transaction mode.

4.1.1 BC/MC R3bc

BC/MC R3bc consists in an online chip transaction, requiring the introduction of his PIN by the cardholder.
On the PDV, it is managed by the *Proprietary Application*.

4.1.2 Proton

Proton is an electronic purse payment scheme consisting in an offline chip transaction, where no PIN introduction is required.
As for BC/MC R3bc, on the PDV, it is managed by the *Proprietary Application*.

4.1.3 EMV (including BC/MC EMV)

An EMV transaction is a chip transaction, currently using the C-TAP protocol to communicate with the acquirer. On the PDV, it is managed by the *Common Application*.
Many transaction brands fall into this category: Maestro, VISA, Mastercard, Sodexo, Edenred, etc. This list also contains the SEPA-compliant version of BC/MC: BC/MC EMV.
Different acquirers may be concerned, depending on the contract the merchant has for the different brands he accepts.

Note: depending on the configuration of the PDV or on the contract the merchant has with the acquirer, some mode or brand may be unavailable.

4.2 vmc_debit_request

With this message, VMC requests the PDV to perform a transaction. Whatever the transaction mode or the payment scheme, the format of the vmc_debit_request message is as follow:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4x	Always **01 07**
-	vic_msg_code	2	2a	Always **56 44** (=VD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
3	vic_tx_amt	3	6x	Amount in eurocent (hexadecimal !). Example : 10,00 EUR -> **00 03 E8** Range : [**00 00 01**..**FF FF FF**]
12	tx_type	1	2i	Always **04** (single transaction)
16	vic_card_ind	1	2i	Always **01** (remove card after transaction)
20	vic_to	2	4x	Time given for card insertion. Always **00 1E** (30")
21	vic_tx_id	4	8x	Transaction identifier issued by VMC. Must be different for every transaction. Range : [**00 00 00 01**..**FF FF FF FF**]
23	curcy	2	4i	Always **09 78** (EURO)
25	vic_cust_ind	1	2i	Always **01** (ask for cardholder validation)
143	operator_nr	2	4i	Identifies the operator of the VMC. Range : [**00 01** .. **99 99**]
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)

After having sent the ***vmc_debit_request***, the VMC should grant the PDV **90** seconds to perform the transaction. If no answer (***pdv_debit_result*** or ***pdv_error***) has been received at expiry of this time-out, the VMC should send a ***vmc_cancel*** for recovery. For full description of this procedure, refer to Section 4.5.2 infra.

NOTE ON LIMITING THE MEANS OF PAYMENT AVAILABLE TO THE CARDHOLDER (*NOT RECOMMENDED*)

The VMC can specify the means of payment to be used by the cardholder (**set only 1 bit**) or limit the means of payment available to the cardholder (**set more bits**) by adding the optional field ***vic_bit_map_application_id***

171	vic_bit_map_application_id	16	128b	limit the means of payment available to cardholder

The VMC should set the bits of the payment types it wants to accept according to the table below :
(table contains only the most frequently used payment types, refer to the full specification for the entire list)

BIT	66	82	114	120	121	122	123	124	125	127	128
PAY TYPE	BC/MC EMV	Visa VPAY	Visa Electron	JCB	Diner's	American Express	Master card	Visa	Proton	Maestro	BC/MC R3bc

NOTE :
The ***vic_bit_map_application_id*** for TINA may not be used, for Emergency fallback (TINA EMV) neither.

Examples of ***vic_bit_map_application_id*** :
to accept only Proton payment : set bit 125 **00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08**
to accept Proton and BC/MC (both R3bc and EMV[1]) payment :
set bit 65, 125,128 **00 00 00 00 00 00 00 00 40 00 00 00 00 00 00 09**
to accept Diner's, Amex & Visa : set bit 121,122,124 **00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 D0**

[1] it is recommended to set both BC/MC bits, in prevision of the phasing out of BC/MC R3bc

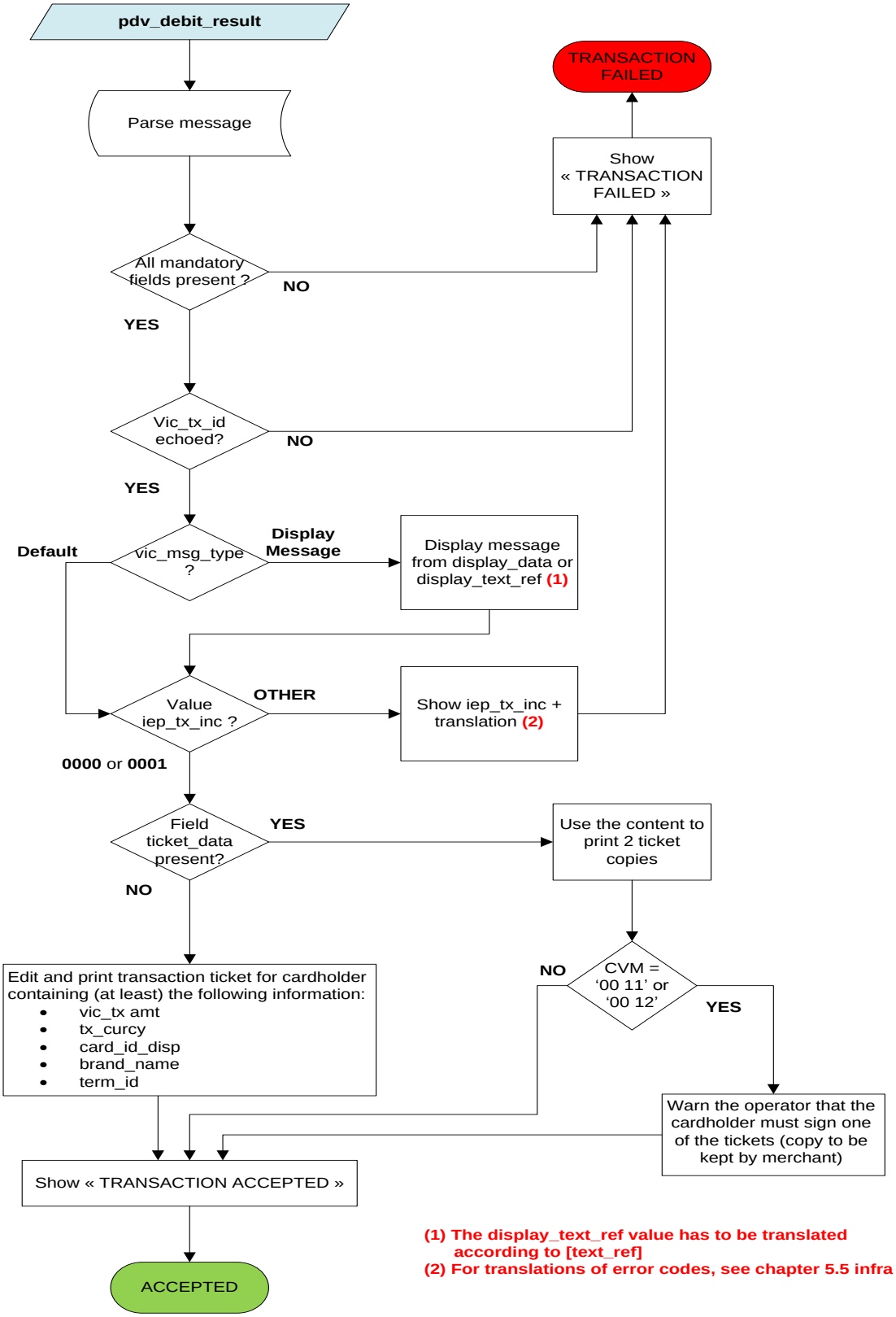
4.3 pdv_debit_result

As already mentioned, in order to perform transactions, the PDV has two different applications on board, they are referred to as “Proprietary application” and “Common application”, each handling different means of payment.

At this time – **but subject to change at any time** – the means of payment are distributed as follows:

- Proprietary application: **BC/MC, TINA, PROTON**.
- Common application: **BC/MC-EMV, TINA EMV, MAESTRO, all other cards**.

The PDV communicates the result of the transaction to the VMC in a ***pdv_debit_result***. The layout of this message is different depending on which application performed the transaction and on which protocol was used between the PDV and the acquirer host.
This implies the way the message has to be decoded, as illustrated by the flowchart below. In particular, the *Common application* provides an already formatted transaction receipt, while the ticket has to be manually constructed in the case of the *Proprietary application*.


pdv_debit_result
Parse message
All mandatory fields present ?
NO
YES
TRANSACTION FAILED
Show « TRANSACTION FAILED »
Vic_tx_id echoed?
NO
YES
Default
vic_msg_type ?
Display Message
Display message from display_data or display_text_ref (1)
Value iep_tx_inc ?
OTHER
Show iep_tx_inc + translation (2)
0000 or 0001
Field ticket_data present?
YES
NO
Use the content to print 2 ticket copies
CVM = '00 11' or '00 12'
NO
YES
Edit and print transaction ticket for cardholder containing (at least) the following information:
vic_tx amt
tx_curcy
card_id_disp
brand_name
term_id
Warn the operator that the cardholder must sign one of the tickets (copy to be kept by merchant)
Show « TRANSACTION ACCEPTED »
ACCEPTED
(1) The display_text_ref value has to be translated according to [text_ref]
(2) For translations of error codes, see chapter 5.5 infra

4.3.1 Layout *pdv_debit_result* of a transaction performed by the Proprietary Application:

FIELD		FORMAT		COMMENTS
index	name	# bytes	length	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 44** (=PD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
2	term_id	8	8a	Terminal number Range : [**00 00 00 00** .. **99 99 99 99**]
3	vic_tx_amt	3	6x	Transaction amount as asked by VMC. Range : [**00 00 00** .. **FF FF FF**]
4	iep_tx_inc	2	4i	Indicates the result of the transaction Range : [**00 00** .. **99 99**]
5	lg_cust	1	2i	Indicates the language of the card used Range : [**00** .. **99**]
7	pdv_clct_nr	2	4i	Back-office information range : [**00 00** .. **09 99**]
8	pdv_gen_tx_nb	4	8x	Back-office information range : [**00 00 00 00** .. **FF FF FF FF**]
9	pdv_state	4	32b	Indicates the 'state' of the PDV. Irrelevant.
11	card_id_disp	24	24a	Contains the card nr, sent in "ready to print" ASCII-format; some digits of the number may be scrambled (depends on acquirer's security rules)
21	vic_tx_id	4	8x	Echo of transaction identifier issued by the VMC. Range : [**00 00 00 01** .. **FF FF FF FF**]
22	vic_msg_type	1	8b	Default **00**. Irrelevant.
23	curcy	2	4i	Always **09 78** (EURO)
125	<tx>vic_tx_amt	3	6x	Transaction amount as debited by PDV. Range : [**00 00 01** .. **FF FF FF**]
126	<tx>curcy	2	4i	Always **09 78** (EURO)
171	vic_bit_map_application_id	16	128b	Indicates PDV-application used for transaction.
178	brand_name	1 + 1..24	2i + 1..24a	The first byte indicates the length of the field, this byte not included. The following bytes contain the brand name. This field identifies the means of payment used by the cardholder.
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)

4.3.2 Layout *pdv_debit_result* of a transaction performed by the Common Application:

FIELD		FORMAT		COMMENTS
index	name	# bytes	length	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 44** (=PD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
2	term_id	8	8a	Terminal identifier
3	vic_tx_amt	3	6x	Transaction amount as asked by the VMC. range: [**00 00 00** .. **FF FF FF**]
4	iep_tx_inc	2	4i	Indicates the result of the transaction range: [**00 00** .. **99 99**]
5	lg_cust	1	2i	Indicates the language of the card used range : [**00** .. **99**]
9	pdv_state	4	32b	Indicates the 'state' of the PDV. Irrelevant.
11	card_id_disp	24	24a	Contains the card nr, sent in "ready to print" ASCII-format; some digits of the number may be scrambled (depends on acquirer's security rules)
21	vic_tx_id	4	8x	Echo of transaction identifier issued by the VMC. Range: [**00 00 00 01** .. **FF FF FF FF**]
22	vic_msg_type	1	8b	Default **00**. If **10**, indicates that a message has to be displayed by the VMC, as given in the field display_message or referenced by the field display_text_ref (depends on the presence of the fields).
23	curcy	2	4i	Always **09 78** (EURO)
36	vic_data	2 + 1..999	4x + 1..1998x	May contain different information from the acquirer. See full spec for format and details. e.g.: acquirer-issued authorisation code for the transaction.
125	<tx>vic_tx_amt	3	6x	Transaction amount as debited by PDV[2]. Range: [**00 00 01** .. **FF FF FF**]
126	<tx>curcy	2	4i	Always **09 78** (EURO)
145	ticket_data	2 + 1..999	4i + 1..999a	The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains data (sent in "ready-to-print" ASCII-format) which **must** be printed by the VMC.
159	display_data	2 + 1..999	4i + 1..999a	This field has to be taken into account only if indicated by the vic_msg_type field. The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains data (sent in "ready-to-print" ASCII-format) that **must** be displayed by the VMC.
170	transaction_protocol	2	4i	Info about communication between PDV and host. Range : [**00 00** .. **00 34**] (1st byte always **00**)
171	vic_bit_map_application_id	16	128b	Indentifies the card application used for the transaction.
172	transaction_identifier	4	8i	Acquirer-issued identifier of the transaction. Range : [**00 00 00 00** .. **99 99 99 99**]
173	date_and_time	7	14i	Time of the transaction[3] Format : **YY MM DD hh mm ss**
174	brand_id	2	4i	Brand_id of the card used. Example : **20 02** (Visa) Range : [**00 00** .. **99 99**]

[2] In the case of Partial Approval (indicated by a iep_tx_inc at **00 01**), this amount can be different than the one specified in the field vic_tx_amt.
[3] as logged by the acquirer, or by the terminal in case of emergency fallback.

176	cvm	2	4i	Info about how cardholder validates the transaction. range : [**00 00** .. **00 22**] (1st byte always **00**) Cardholder must sign the merchant copy of the transaction ticket only if cvm = xx**1**x.
178	brand_name	1 + 1..24	2i + 1..24a	The first byte indicates the length of the field, this byte not included. The following bytes contain the brand name. This field identifies the means of payment used by the cardholder.
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)
182	display_text_ref	2	4x	This field has to be taken into account only if indicated by the vic_msg_type field. Contains a reference to a message to be displayed by the VMC, as defined in [text_ref].

4.4 pdv_error

In some exceptional cases, after having received a ***vmc_debit_request*** from the VMC, the PDV answers with a ***pdv_error*** instead of a ***pdv_debit_result***.

If this is the case, the VMC must conclude that the transaction has failed, display the error code as well as a translation of this error code to the operator. See 4.6 infra for incident code translations.

Layout:

FIELD		OBSERVATIONS	
index	name	field length	remarks
-	vic_protocol_id	2	Always **01 07**
-	vic_msg_code	2	Always **50 45** (=PE)
-	vic_bit_map	24	See explanation in 3.1 supra
4	iep_tx_inc	2	Indicates the error code. Range : [**00 00** .. **99 99**]
9	pdv_state	4	Indicates the 'state' of the PDV. Irrelevant.
22	vic_msg_type	1	Always **00**. Irrelevant
179	vic_version	1	Always **13** (vic_protocol_subversion 1.07/13)

4.5 vmc_cancel

Layout:

FIELD		OBSERVATIONS	
index	name	field length	remarks
-	vic_protocol_id	2	Always **01 07**
-	vic_msg_code	2	Always **56 43** (=VC)
-	vic_bit_map	24	See explanation in 3.1 supra
31	vmc_type	3	Must always be **00 00 00**
179	vic_version	1	Must always be **13** (vic_protocol_subversion 1.07/13)

Use:

4.5.1 vmc_cancel at start-up

In order to assure that the communication with the PDV works correctly before performing transactions, the VMC should send this message after every reboot of the VMC. The PDV replies with the last ***pdv_debit_result*** or ***pdv_error*** it has sent; the content of this reply must be ignored by the VMC.

4.5.2 vmc_cancel for transaction recovery

After having sent the ***vmc_debit_request***, the VMC should grant the PDV **90** seconds to perform the transaction. If no answer (***pdv_debit_result*** or ***pdv_error***) has been received after these **90** seconds, the VMC should send a ***vmc_cancel*** for recovery.

Upon receiving a ***vmc_cancel*** message, the PDV will resend – maximum **15 seconds** later - the last ***pdv_debit_result*** it has sent. (remark : the field **vic_msg_type** will contain **80** instead of **00**, because this is a duplicate message)

Upon receiving a ***pdv_debit_result***, the VMC should check the field **vic_tx_id** :

- If it contains the same value as the one used in the ***vmc_debit_request***, the VMC must analyse the ***pdv_debit_result*** as usual and can conclude if the transaction was successful or not.
- If it contains a different value for **vic_tx_id**, the VMC must conclude that the transaction failed.

Exceptional situations after ***vmc_cancel*** for recovery:

- 15 seconds after the ***vmc_cancel***, no ***pdv_debit_result*** is received at all : the result of the transaction is unsure. The operator has to be informed and the transaction has to be checked manually[4].
- Instead of a ***pdv_debit_result***, a ***pdv_error*** with the field **iep_tx_inc = 4009** is received. The result of the transaction is uncertain. The operator has to be informed and the transaction has to be checked manually[5].

[4] This can be done:
- by reinserting/re-swiping any card in the PDV and pressing <MENU>
- by the terminal MENU => STOP-OK-MENU-2. LAST TRANSACTION
- by phoning the acquirer.

[5] idem

4.6 Error codes - translations

If a transaction ends with an error:

- the VMC receives a ***pdv_debit_result*** message with **iep_tx_inc** <> "0000" or "0001"
  or
- the VMC receives a ***pdv_error*** message with **iep_tx_inc** <> "0000" or "0001"

The VMC must display :

- An easy-to-understand translation of the error code
- The error code that was received

Example: Upon receiving error code 4002, the VMC must display

```
TIME-OUT CUSTOMER
NO VALIDATION
(4002)
```

CODE	ENGLISH	NEDERLANDS	FRANCAIS
2001	No more paper In the terminal printer	Geen papier meer in printer	Plus de papier dans l'imprimante
2002	Mandatory ticket pending	Verplicht ticket nog hangende	Ticket obligatoire En attente
4001	Time-out customer No card inserted	Time-out klant Geen kaart ingebracht	Time-out client Pas d'insertion de carte
4002	Time-out customer No validation	Time-out klant Geen bevestiging	Time-out client Pas de confirmation
4005	Balance too low	Saldo ontoereikend	Solde insuffisant
4006	Bad value in field	Ongeldige waarde	Mauvaise donnée
4008	Invalid message	Ongeldige boodschap	Message invalide
4009	Terminal not ready Uncertain transaction	Terminal niet klaar Onzekere transactie	Terminal pas prêt Transaction incertaine
4011	Time-out customer on card reading retry	Time-out klant op kaart herlezen	Time-out client sur relecture carte
5001	Technical problem	Technisch probleem	Problème technique
5002	Transaction refused by the host	Transactie geweigerd door de host	Transaction refusée par le host
5003	Duplicate transaction	Dubbele transactie	Double transaction
5004	Technical problem on host	Technisch probleem op de host	Problème technique sur le host
5006	Stop customer	Stop klant	Stop client
5008	Invalid currency	Ongeldige munteenheid	Devise incorrect
5009	Transaction refused by the terminal	Transactie geweigerd door terminal	Transaction refusée par le terminal
5010	Communication problem	Communicatieprobleem	Problème de communication
5012	Limit Exceeded	Limiet overschreden	Limite dépassée
5013	Transaction refused by the terminal	Transactie geweigerd door terminal	Transaction refusée par le terminal
5014	Transaction refused by the card	Transactie geweigerd door de kaart	Transaction refusée par la carte
5024	Problem linked to the card	Probleem kaart	Problème lié à la carte
5025	Card blocked	Kaart geblokkeerd	Carte bloquée
5026	Card not allowed	Kaart geweigerd	Carte non autorisée
5027	Incorrect PIN	PIN onjuist	Pin incorrect

5028	Product not allowed	Product niet toegelaten	Produit non autorisé
5029	Product not available	Product niet beschikbaar	Produit non disponible
5030	Card information not available	Kaart informatie niet beschikbaar	Information carte non disponible

All other error codes must result in a display as shown below:

```
TECHNICAL PROBLEM
(XXXX)
```

(Where **XXXX** is the ***iep_tx_inc*** received)

5. OTHER FUNCTIONALITIES

5.1 Special credit card operations

5.1.1 Sale cancel (00)

A sale can be cancelled using the ***vmc_debit_request*** below.
The card used for the sale has to be re-read on the PDV, the amount has to be the same.
The time permitted between the sale and the sale cancel depends on the acquirer.

The VMC needs to know the authorisation code (ACODE) of the transaction it wants to cancel and has to send it to the PDV in the field **vic_data** of the ***vmc_debit_request***.
(This code was sent to the VMC in the field **vic_data** of the ***pdv_debit_result*** message of the sale)

vmc_debit_request to be used
Transaction amount : **EUR 0,10**
ACODE : **123456**

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4x	Always **01 07**
-	vic_msg_code	2	2a	Always **56 44** (=VD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
3	vic_tx_amt	3	6x	Amount in eurocent (hexadecimal !). Example : 0,10 EUR → **00 00 0A** Range : [**00 00 01**..**FF FF FF**]
12	tx_type	1	2i	Always **04** (single transaction)
16	vic_card_ind	1	2i	Always **01** (remove card after transaction)
20	vic_to	2	4x	Time given for card insertion. Always **00 1E** (30")
21	vic_tx_id	4	8x	Transaction identifier issued by VMC. Must be different for every transaction. Range : [**00 00 00 01**..**FF FF FF FF**]
23	curcy	2	4i	Always **09 78** (EURO)
25	vic_cust_ind	1	2i	Always **01** (ask for cardholder validation)
36	vic_data	4 + 6	8i + 6a	First bytes always **00 08 07 06**. Last bytes contain the authorization code. Example: **00 08 07 06 31 32 33 34 35 36**
143	operator_nr	2	4i	Identifies the operator of the VMC. Range : [**00 01** .. **99 99**]
144	function_cc	2	2a	Always **30 30** (= '00', sale cancel)
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)

5.1.2 Mail order sale

This functionality can only be used if permitted by the acquirer.
This is a sale using manual entry of the card data, used if cardholder not present (e.g. telesales).

vmc_debit_request to be used

card number : 1234 5678 9012 3456
expiration date : 12 10 (December 2010)
cvc-code: 123

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4x	Always **01 07**
-	vic_msg_code	2	2a	Always **56 44** (=VD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
3	vic_tx_amt	3	6x	Amount in eurocent (hexadecimal !). Example : 0,10 EUR → **00 00 0A** Range : [**00 00 01**..**FF FF FF**]
12	tx_type	1	2i	Always **04** (single transaction)
16	vic_card_ind	1	2i	Always **01** (remove card after transaction)
20	vic_to	2	4x	Time given for card insertion. Always **00 1E** (30")
21	vic_tx_id	4	8x	Transaction identifier issued by VMC. Must be different for every transaction. Range : [**00 00 00 01**..**FF FF FF FF**]
23	curcy	2	4i	Always **09 78** (EURO)
25	vic_cust_ind	1	2i	Always **01** (ask for cardholder validation)
142	iso2_track	19	38x	Iso2 track field e.g.: **5D 12 34 56 78 90 12 34 56 D1 21 00 00 00 00 00 00 00 00**
143	operator_nr	2	4i	Identifies the operator of the VMC. Range : [**00 01** .. **99 99**]
177	cvc2	2	4i	Card validation code 2 e.g.: **01 23**
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)

5.2 Proton configuration

Next to the transaction mechanism of Proton (described among others in Section 4), the accumulated "electronic money" has to be transferred from the terminal to the service provider's bank account. The way this transfer is done can be configured with the aid of the following **vmc_command** message:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **C0 00** (configuration setting)
18	vic_config_clct_amt	4	8x	Proton transfer settings. Following values are possible : **00 00 00 00**: Proton transfer is managed entirely by PDV. [**00 00 00 01..00 0F FF FF**]: When this amount (hexadecimal eurocents) is reached, the PDV will execute Proton transfer in the following night. **FF FF FF FF**: PDV executes Proton transfer ONLY upon VMC request.
19	pdv_state_mask	4	32b	Always **00 10 00 00**. (=bit 12 set)(PDV will send unsollicited message upon **proton transfer start/proton transfer end**)
23	curcy	2	4i	Always **09 78** (EUR)
171	vic_bit_map_application_id	16	128b	Always **00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

The following ***pdv_command_reply*** will then be received if everything went wel:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident) If another value is received, configuration was unsuccessful.
171	vic_bit_map_application_id	16	128b	Always **00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

Depending on the configuration, unsolicited ***pdv_state_info*** message may be then received:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 53** (=PS)
-	vic_bit_map	8	64b	see explanation in chapter 3.1 supra.
4	iep_tx_inc	2	4i	Always **00 00**
9	pdv_state	4	32b	Indicates the state of the PDV. - bit 2 indicates card reader status - bit 12 indicates proton transfer status. - other bits are irrelevant and should not be interpreted.

				Example : **40 00 00 00** : indicates a card is in the reader.
22	vic_msg_type	1	8b	**40** when this message is sent unsollicited. **00** when this message is sent in reply to vmc_state_request.
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

5.3 TINA

TINA is an optional service for BC/MC R3bc chip card transactions. It enables the terminal to perform fully offline transactions if the host is unable to approve them online (e.g. if unreachable).
This service has to be manually activated by the merchant, while its deactivation is managed automatically by the PDV (a manually deactivation is however available as described below).

Note that this service is not available on every PDV, as it depends on the contracts the merchant has with the acquirer.

5.3.1 TINA Activation

TINA is activated by exchanging a **vmc_command/pdv_command_reply** message.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 00** (Service activation request)
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident)
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If the field ***iep_tx_inc*** <> 00 00, TINA-activation did not succeed. Possible causes:
iep_tx_inc 5016: Too many TINA-transactions were already executed this month.
iep_tx_inc 5017: Too many TINA-transaction logs are present in the terminal.
iep_tx_inc 5018: TINA already activated.
iep_tx_inc 5020: TINA activation not permitted by the acquirer.
iep_tx_inc 5021: Too many TINA-transaction records are present in the terminal.
iep_tx_inc 5022: Too many TINA-activations were already executed this month.

Please note:
To avoid unnecessary activations, the first transaction after activation will be an online try.
If the PDV can perform a certain amount (configured by the acquirer) of successful online transactions in a row, TINA will be deactivated automatically.

5.3.2 TINA Transaction

vmc_debit_request to be used

The ***vmc_debit_request*** is a normal one, as described in Section 4.2; no reference to TINA should be made in it, as the ECR is not supposed to know whether the TINA service will be used or not for a given transaction.

pdv_debit_result to be used

Examples of message lay-out can be found infra (see Section 5.3.5).

Specific TINA-incidents:
iep_tx_inc 5016: Too many TINA-transactions were already executed this month.
iep_tx_inc 5021: Too many TINA-transaction records are present in the terminal.
iep_tx_inc 5023: Amount too high (amount of a TINA-transaction is limited to EUR 500)
iep_tx_inc 5024: TINA-transaction impossible due to a problem on the card.

5.3.3 TINA Deactivation

Please note:
When performing TINA transactions, the PDV will try an online transaction after a grace period (configured by the acquirer).
If the PDV can perform a certain amount (configured by the acquirer) of successful online transactions in a row, TINA will be deactivated automatically.
The manual deactivation is then available as optional mechanism.

TINA is de-activated by exchanging a ***vmc_command***/***pdv_command_reply*** message.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 01** (Service deactivation request)
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident)
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If the field ***iep_tx_inc*** <> 00 00, TINA-deactivation did not succeed. Possible cause:
iep_tx_inc 5019: TINA already deactivated.

5.3.4 TINA Consultation

The PDV can exchange the ***vmc_command/pdv_command_reply*** messages below to find out if TINA is active or not.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 02** (Service consultation request)
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	**50 18** or **50 19**
171	vic_bit_map_application_id	16	128b	TINA identifier: **00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If ***iep_tx_inc*** = 5018: TINA active
If ***iep_tx_inc*** = 5019: TINA inactive

5.3.5 TINA Back-up

The PDV keeps all TINA-transactions in its non-volatile memory in order to avoid losing them due to a reboot of the PDV. However, loss of transaction data remains possible (e.g. terminal crash). A back-up system is needed to avoid loss of TINA-transactions (loss of money for the merchant).

Three possibilities exist, of which one **must** be available on the VMC:

1. Printing out the transaction data on the VMC ticket printer and keep these tickets.
2. Stocking the transaction data in a file kept on the VMC.
3. Keeping the tickets printed by the PDV.(if PDV is equipped with printer).

5.3.5.1 Printing out the transaction data on the VMC-ticket printer and keep those tickets.

The TINA-transaction-data-ticket must be printed separately from all other tickets and **may not be given to the cardholder**. It must be kept by the merchant. The VMC may not change the layout of the ticket.

As an example, you will find below the expected look of a TINA-transaction-ticket as well as the **pdv_debit_result** from which the contents of the ticket were taken:

```
TERMINAL : 01111113 (1)
PERIOD : 69 (2)
TRANSACTION : 1 (3)

            TINA (9)

CARD : 059-9000121-95 0311 (4)
TIME : 06-10-2003 16:14 (8)
AMNT : EUR 0,04 (6) (5)

R3D  : 0502 000D 4009 7810
       1231 0100 0002 0000
       0004 8F0B 8A85 4BF9
       1B6B (7)
KECS : 2D77C952 (7)
```

vic_protocol_id	01 07
vic_msg_code	50 44
vic_bit_map	FB A0 0E 00 00 00 00 00 80 00 00 00 00 00 00 0C 00 00 80 00 00 28 60 00
term_id	30 31 31 31 31 31 31 33 **(1)**
vic_tx_amt	00 00 04
iep_tx_inc	00 00
lg_cust	01
pdv_clct_nr	00 69 **(2)**
pdv_gen_tx_nr	00 00 00 01 **(3)**
pdv_state	E0 00 00 00
card_id_disp	30 35 39 2D 39 30 30 30 31 32 31 2D 39 35 20 30 33 31 31 20 20 20 20 20 **(4)**
vic_tx_id	00 12 34 56
vic_msg_type	00
curcy	09 78
<tx>vic_tx_amt	00 00 04 **(5)**
<tx>curcy	09 78 **(6)**
ticket_data	01 06 52 33 44 3A 20 30 35 30 32 20 30 30 30 44 20 34 30 30 39 20 37 38 31 30 0D 0A 20 20 20 20 20 31 32 33 31 20 30 31 30 30 20 30 30 30 32 20 30 30 30 30 0D 0A 20 20 20 20 20 30 30 30 34 20 38 46 30 42 20 38 41 38 35 20 34 42 46 39 0D 0A 20 20 20 20 20 31 42 36 42 20 0D 0A 4B 45 43 53 3A 20 32 44 37 37 43 39 35 32 0D 0A **(7)**
vic_bm_app_id	00 00 00 00 00 00 00 00 00 00 00 00 10 00 00 00
date_and_time	20 03 10 06 16 14 00 **(8)**
brand_name	04 54 49 4E 41 **(9)**
vic_version	13

5.3.5.2 Stocking the transaction data in a file kept on the VMC

As an example, we will build a file concerning the two TINA-transactions from which the **pdv_debit_result** messages are shown below. The data needed for the file are highlighted.

1st transaction :

vic_protocol_id	01 07
vic_msg_code	50 44
vic_bit_map	FB A0 0E 00 00 00 00 00 80 00 00 00 00 00 00 0C 00 00 80 00 00 28 60 00
term_id	30 31 37 30 35 30 31 39
vic_tx_amt	00 00 08
iep_tx_inc	00 00
lg_cust	01
pdv_clct_nr	00 01
pdv_gen_tx_nr	00 00 00 01
pdv_state	E0 00 00 00
card_id_disp	32 39 33 2D 39 30 30 30 30 30 35 2D 31 32 20 33 31 30 33 20 20 20 20 20
vic_tx_id	00 00 00 FB
vic_msg_type	00
curcy	09 78
<tx>vic_tx_amt	00 00 08
<tx>curcy	09 78
ticket_data	01 06 52 33 44 3A 20 30 35 30 31 20 30 33 30 37 20 31 38 30 39 20 37 38 30 39 0D 0A 20 20 20 20 20 31 32 33 31 20 30 31 30 30 20 30 30 30 39 20 30 30 30 34 0D 0A 20 20 20 20 20 33 45 30 34 20 32 44 45 41 20 31 38 34 37 20 35 33 44 44 0D 0A 20 20 20 20 20 46 36 31 39 20 0D 0A 4B 45 43 53 3A 20 38 45 32 37 30 45 36 38 0D 0A
vic_bm_app_id	00 00 00 00 00 00 00 00 00 00 00 10 00 00 00
date_and_time	20 04 03 08 15 25 00
brand_name	04 54 49 4E 41
vic_version	13

2nd transaction :

vic_protocol_id	01 07
vic_msg_code	50 44
vic_bit_map	FB A0 0E 00 00 00 00 00 80 00 00 00 00 00 00 0C 00 00 80 00 00 28 60 00
term_id	30 31 37 30 35 30 31 39
vic_tx_amt	00 00 64
iep_tx_inc	00 00
lg_cust	01
pdv_clct_nr	00 01
pdv_gen_tx_nr	00 00 00 02
pdv_state	E0 00 00 00
card_id_disp	32 39 33 2D 39 30 30 30 30 30 35 2D 31 32 20 33 31 30 33 20 20 20 20 20
vic_tx_id	00 00 00 FC
vic_msg_type	00
curcy	09 78
<tx>vic_tx_amt	00 00 64
<tx>curcy	09 78
ticket_data	01 06 52 33 44 3A 20 30 35 30 31 20 30 33 30 37 20 31 38 30 39 20 37 38 30 39 0D 0A 20 20 20 20 20 31 32 33 31 20 30 31 30 30 20 30 30 30 39 20 30 30 30 34 0D 0A 20 20 20 20 20 33 45 30 34 20 38 36 38 35 20 30 41 37 31 20 39 38 38 45 0D 0A 20 20 20 20 20 33 35 44 46 20 0D 0A 4B 45 43 53 3A 20 31 36 35 35 35 33 45 39 0D 0A
vic_bm_app_id	00 00 00 00 00 00 00 00 00 00 00 10 00 00 00
date_and_time	20 04 03 08 15 26 22
brand_name	04 54 49 4E 41
vic_version	13

Filename:
The filename should be: **TINA.T01705019.D20040308.BIN**
The following DOS-format is also acceptable: **70501908.BIN** (terminal_id(6) + day(2).BIN)

Record 00:
Record type: **00**
Data Length: **00 4F** (79 bytes)
Dat :
File identification: **54 49 4E 41 2E 54 30 31 37 30 35 30 31 39 2E 44 32 30 30 34 30 33 30 38 2E 42 49 4E 00 00 00

5.4 TINA EMV

TINA EMV, or *Emergency fallback*, is an optional service for EMV transactions (BC/MC EMV, Maestro, VISA, Mastercard, etc.). As for the normal TINA, it enables the terminal to perform fully offline transactions if the host is unable to approve them online (e.g. if unreachable).
This service has to be manually activated, while its deactivation is managed automatically (a manually deactivation is however available as described below).

Note that this service is not available on every PDV or for every brands, as it depends on the contracts the merchant has with the acquirer(s).

5.4.1 TINA EMV Activation

The Emergency fallback is activated by exchanging a ***vmc_command*** and a ***pdv_command_reply*** messages.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 00** (Service activation request)
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident)
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If the field ***iep_tx_inc*** <> 00 00, TINA EMV-activation did not succeed. Possible causes:
iep_tx_inc 5016: Too many Emergency fallback transactions were already executed this month.
iep_tx_inc 5017: Too many Emergency fallback transaction logs are present in the terminal.
iep_tx_inc 5018: Emergency fallback already activated.
iep_tx_inc 5020: Activation not permitted by any acquirer.

Please note:
To avoid unnecessary activations, the first transaction after activation will be an online try.
If the PDV can perform a certain amount (configured by the acquirer) of successful online transactions in a row, the Emergency fallback will be deactivated automatically.

5.4.2 TINA EMV Transaction

A transaction is performed by exchanging a ***vmc_debit_request*** and a ***pdv_debit_result*** messages.

vmc_debit_request to be used

The ***vmc_debit_request*** is a normal one, as described in Section 4.2; no reference to the *Emergency fallback* should be made in it, as the ECR is not supposed to know whether the fallback will be used or not for a given transaction.

pdv_debit_result to be used

The ***pdv_debit_result*** sent by the PDV in the case the *Emergency fallback* has been required is a special case of the one used for a *Common Application*, Section 4.3.2, completed by 4 other fields necessary for the recovery (`pdv_clct_nr`, `vic_proprietary_data`, `acquirer_id` and `tsc_id`):

FIELD		FORMAT		COMMENTS
index	name	# bytes	length	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 44** (=PD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
2	term_id	8	8a	Terminal identifier
3	vic_tx_amt	3	6x	Transaction amount as asked by VMC. range: [**00 00 00** .. **FF FF FF**]
4	iep_tx_inc	2	4i	Indicates the result of the transaction range: [**00 00** .. **99 99**]
5	lg_cust	1	2i	Indicates the language of the card used range : [**00** .. **99**]
7	pdv_clct_nr	2	4i	*Emergency fallback* Batch number, see Section 5.4.5 range: [**00 00** .. **09 99**]
9	pdv_state	4	32b	Indicates the 'state' of the PDV. Irrelevant.
11	card_id_disp	24	24a	Contains the card nr, sent in "ready to print" ASCII-format ; some digits of the number may be scrambled (depends on acquirer's security rules)
21	vic_tx_id	4	8x	Echo of transaction identifier issued by the VMC. Range: [**00 00 00 01** .. **FF FF FF FF**]
22	vic_msg_type	1	8b	Default **00**. If **10**, indicates that a message has to be displayed by the VMC, as given in the field display_message or referenced by the field display_text_ref (depends on the presence of the fields).
23	curcy	2	4i	Always **09 78** (EURO)
35	vic_proprietary_data	2 + 1..999	4i + 1..1998x	The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains data enabling the recovery, as described in Section 5.4.5.
36	vic_data	10	8x + 6a	First 4 bytes always **00 08 07 06**. Last 6 bytes show acquirer-issued authorisation code for the transaction.
125	<tx>vic_tx_amt	3	6x	Transaction amount as debited by the PDV. Range: [**00 00 01** .. **FF FF FF**] Should be equal to vic_tx_amt
126	<tx>curcy	2	4i	Always **09 78** (EURO)
145	ticket_data	2 + 1..999	4i + 1..999a	The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains data (sent in "ready-to-print" ASCII-format) which **must** be printed by the VMC.
159	display_data	2 + 1..999	4i + 1..999a	This field has to be taken into account only if indicated by the vic_msg_type field.

				The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains data (sent in "ready-to-print" ASCII-format) that **must** be displayed by the VMC.
167	acquirer_id	4	8i	Identifier of the acquirer range: [**00 00 00 00** .. **99 99 99 99**]
170	transaction_protocol	2	4i	Must be **00 14** (chip, *Emergency fallback*)
171	vic_bit_map_application_id	16	128b	Indicates the Emergency fallback (TINA EMV) has been used (bit 65 set) along with the PDV-application used for transaction (double information).
172	transaction_identifier	4	8i	Acquirer-issued identifier of the transaction. Range : [**00 00 00 00** .. **99 99 99 99**]
173	date_and_time	7	14i	Time of the transaction as logged by the acquirer. Format : **YY MM DD hh mm ss**
174	brand_id	2	4i	Brand id of the card used. Example : **20 02** (Visa) Range : [**00 00** .. **99 99**]
176	cvm	2	4i	Info about how cardholder validates the transaction. range : [**00 00** .. **00 22**] (1st byte always **00**) Cardholder must sign the merchant copy of the transaction ticket only if cvm = xx**1**x.
178	brand_name	1 + 1..24	2i + 1..24a	The first byte indicates the length of the field, this byte not included. The following bytes contain the brand name. This field identifies the means of payment used by the cardholder.
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)
182	display_text_ref	2	4x	This field has to be taken into account only if indicated by the vic_msg_type field. Contains a reference to a message to be displayed by the VMC, as defined in [text_ref].
183	tsc_id	6	12x	Identifies the TSC with respect to the configured security scheme used for the transaction, if imposed by acquirer.

Note: after an Emergency fallback, as opposed to normal ***pdv_debit_result*** messages, the `vic_bit_map_application_id` field contains a double information: the bit identifying the PDV-application used for the transaction is set as normally, but, in addition, the 65th bit (Emergency fallback (TINA EMV)) is also set, in order to indicate that the transaction is performed in Emergency fallback.

5.4.3 TINA EMV Deactivation

Please note:
When performing Emergency fallback transactions, the PDV will try an online transaction after a grace period (configured by the acquirer).
If the PDV can perform a certain amount (configured by the acquirer) of successful online transactions in a row, the Emergency fallback will be deactivated automatically.
The manual deactivation is then available as optional mechanism.

The Emergency fallback can be deactivated by exchanging a ***vmc_command*** and a ***pdv_command_reply*** messages.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 01** (Service deactivation request)
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident)
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If the field ***iep_tx_inc*** <> 00 00, *Emergency fallback* deactivation did not succeed.
Possible cause:
iep_tx_inc 5019: Emergency fallback already deactivated for all brands.

5.4.4 TINA EMV Consultation

The state of the Emergency fallback can be consulted by exchanging a ***vmc_command*** and a ***pdv_command_reply*** messages.

vmc_command to be used

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	Always **E0 02** (Service consultation request)
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

pdv_command_reply sent by the PDV

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	**50 18** or **5019**
171	vic_bit_map_application_id	16	128b	Emergency fallback (TINA EMV) identifier: **00 00 00 00 00 00 00 00 80 00 00 00 00 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

If ***iep_tx_inc*** = 5018: Emergency fallback still activated for at least one brand
If ***iep_tx_inc*** = 5019: Emergency fallback deactivated for all brands

5.4.5 TINA EMV Recovery

As for TINA, the PDV keeps all Emergency fallback-transactions in its non-volatile memory in order to avoid losing them due to a reboot of the PDV. However, loss of transaction data remains possible (e.g. terminal crash). A recovery system (or "backup of the backup") is needed to avoid loss of transactions (loss of money for the merchant).

The *recovery* mechanism consists in transmitting to an external equipment (e.g. an ECR) the required information necessary to be able to perform a recovery of the Emergency fallback transactions in the case of a terminal crash.

For transactions that are intended for Atos Worldline as acquirer (or processor), the required information and the way it has to be used are described in [recovery_file].
In accordance with that document, in the case of VIC, the required information is transmitted in an adapted ***pdv_debit_result*** message: if the `transaction_protocol` and `vic_bit_map_application_id` fields of the ***pdv_debit_result*** indicates that the Emergency fallback has been used for approving the transaction, the message must be parsed in order to store the content of the `vic_proprietary_data` field (no decoding is required) together with the content of the corresponding field of other transactions for which the following fields are identical:

- `term_id`: the terminal identifier;
- `pdv_clct_nr`: the batch number to which belongs the performed transaction;
- `acquirer_id`: the identifier of the acquirer;
- `tsc_id`: the security scheme TSC identifier (optional, will be present only if the TSC security scheme is imposed by the acquirer).

This information must then be concatenated and transmitted to the corresponding acquirer as described in [recovery_file].

5.5 Partial approval

The *Partial approval* is a specific transactional behaviour of a transaction. It is supported by some acquirers, for some brands only.
It is intended for transactions with a higher amount than the available one on the account linked to the card. Typically, it could be used by some meal voucher issuers (e.g. Sodexo, Edenred) for partially authorizing the transaction, only for the remaining amount of the meal vouchers of the customer.

The *Partial approval* impacts only the ***pdv_debit_response***; the ***vmc_debit_request*** remains indeed unchanged as it is not supposed to be known in advance that the transaction cannot be approved entirely.
If the transaction is only partially approved, the `iep_tx_inc` will be **00 01;** the approved amount will be given in the `<tx>vic_tx_amt` field, while the field `vic_tx_amt` will contain the original requested amount.

As an example, let's consider a (requested) transaction of **EUR 15,00**, being paid with electronic meal vouchers; however, the customers has only **EUR 10,00** left on his associated account. The acquirer approves it then only partially, for **EUR 10,00**. The ***pdv_debit_result*** to be received will look like the following:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4x	Always **01 07**
-	vic_msg_code	2	2a	Always **50 44** (=PD)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
3	vic_tx_amt	3	6x	Amount in eurocent (hexadecimal !). Example : 15,00 EUR → **00 05 DC**
4	iep_tx_inc	2	4i	**00 01** (transaction partially approved).
...	...	...	...	...
23	curcy	2	4i	Always **09 78** (EURO)
125	<tx>vic_tx_amt	3	6x	Partialy approved amount in eurocent (hexadecimal !). Example : 10,00 EUR → **00 03 E8**
126	<tx>curcy	2	4i	Always **09 78** (EURO)
...	...	...	...	...

5.6 Tokenization

The tokenization is the process of creating a *token* on the basis of the card information. This *token* enables to identify a card, without having to transmit or store sensible information (like the PAN), and, doing so, to remain compliant with the security rules.

The *token* can be requested during a transaction by adding the following optional field in the ***vmc_debit_request***:

181	additional_info_bit_map	4	32b	Request for a card token: **80 00 00 00**

In the response (***pdv_debit_result***), the following optional field should be present:

180	card_token	2 + 1..999	4i + 1..1998x	A token generated by the terminal, representing the card uniquely

A *token* might be required before starting the transaction (before sending the ***vmc_debit_request***). Therefore, a ***vmc_purse_request*** message may be used:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 50** (=VP)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
16	vic_card_ind	1	2i	Always **00** (no card withdrawal requested after transaction)
20	vic_to	2	4x	Time given for card insertion (in seconds). Range: [**00 14**..**00 2D**]
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)
181	additional_info_bit_map	4	32b	Request for a card token: **80 00 00 00**

As a response, a ***pdv_purse_result*** message should be received:

FIELD		FORMAT		COMMENTS
index	name	# bytes	length	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 50** (=PP)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	Indicates the result of the transaction Range : [**00 00** .. **99 99**].
5	lg_cust	1	2i	Indicates the language of the card used Range : [**00** .. **99**]
9	pdv_state	4	32b	Indicates the 'state' of the PDV. Irrelevant. range: [**00 00 00 00** .. **FF FF FF FF**]
10	pur_bal	3	6x	Indicates the amount of electronic money in the Purse of the card used, in the currency of the curcy field Range: **[00 00 00** .. **FF FF FF**]
11	card_id_disp	24	24a	Contains the card nr, sent in "ready to print" ASCII-format ; some digits of the number may be scrambled (depends on acquirer's security rules)
23	curcy	2	4i	Always **09 78** (EURO)
171	vic_bit_map_application_id	16	128b	Indicates PDV-application used. Irrelevant.
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)
180	card_token	2 + 1..999	4i + 1..1998x	A token generated by the terminal, representing the card uniquely

If the terminal is not able to generate the requested token (missing keys, for instance), the incident code 5030 (Card information not available) will be returned.

5.7 Counters consultation and counters closing

For each transaction performed, the corresponding acquirer updates some counters. Such counters are associated to a terminal or to a site of several terminals, and are split into brands and/or services.

Some acquirers provide the possibility of consulting and/or manually closing (also called balance) the counters.

The counters consultation, if supported by the acquirer, enables the merchant to consult the counters at any time.
In the same way, the counters closing, if supported by the acquirer, enables the merchant to manually close the counters (and start a new period), at a time that suits him better.

Note: a closing returns the same information than a consultation, but performs the closing for the period at the same time.

The request is addressed to an acquirer in particular, and can concern the requesting terminal or the complete site to which it belongs (all the terminals located on this site).
Note that depending on the implementation at acquirer side, a consultation/closing requested for a terminal can result in the consultation/closing of the entire site.

With VIC, the request for consultation and closing can be made with the following ***vmc_command***:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 4D** (=VM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
17	vic_cmd_code	2	4x	**B0 00**: consultation for terminal only **B0 01**: consultation for complete site **D0 00**: closing for terminal only **D0 01**: closing for complete site
167	acquirer_id	4	8i	Identifier of the acquirer range: [**00 00 00 00** .. **99 99 99 99**]
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

As a response, the following ***pdv_command_reply*** will be received:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **50 4D** (=PM)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
4	iep_tx_inc	2	4i	should be **00 00** (no incident) **50 01** (technical problem) if the request is not supported for the given acquirer or a cloture has been requested for an empty period
157	counter_tab_pa	2 + 1..999	4i + 1..1998x	structured field containing the data of the different counters data; see the complete VIC specification for more details on how to parse it not present if iep_tx_inc different from **00 00**
167	acquirer_id	4	8i	Identifier of the acquirer

				range: [**00 00 00 00** .. **99 99 99 99**]
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

5.8 Belgian ID card reading

The Belgian ID card (eID) reading is an application limited to the Belgian market. It requires the presence of the corresponding application on the terminal, which is not present by default.

In order to identify a Belgian eID card (and to request the introduction of the eID in the card reader), the ECR can send a ***vmc_purse_request*** as follow:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	Always **56 50** (=VP)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
16	vic_card_ind	1	2i	Always **00** (no card withdrawal requested after transaction)
20	vic_to	2	4x	Time given for card insertion (in seconds). Range: [**00 14**..**00 2D**]
179	vic_version	1	2i	Always **13** (vic_protocol_subversion 1.07/13)

In the response ***pdv_purse_result***, the `vic_bit_map_application_id` should identify the EIC (Electronic Identity Card) application:

...	...	...	...	...
171	vic_bit_map_application_id	16	128b	EIC application identifier: **00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 00**
...	...	...	...	...

Once the eID is identified by the terminal, the ECR can communicate with it directly by exchanging ***vmc_data*** and ***pdv_data*** messages with the terminal, containing pure EID ISO 7816 APDU messages in the `vic_data` field. This enables to be independant of the different card releases and card content structures, and to be able to use the governement free middleware. The content of these APDUs is out of scope of this document; please refer to appropriate documentation.

After having received the first APDU command encapsulated in a ***vmc_data***, the EID application gets the hand on the terminal. Then, any message addressed to the EIC application (via the `vic_bit_map_application_id`) other than the ***vmc_data*** will be replied with a ***pdv_error*** (4009 – bad protocol state).

Both ***vmc_data*** and ***pdv_data*** messages have the same layout:

FIELD		FORMAT		COMMENTS
index	**name**	**# bytes**	**type**	
-	vic_protocol_id	2	4i	Always **01 07**
-	vic_msg_code	2	2a	**56 4F** (=VO, ***vmc_data***) or **50 4F** (=PO, ***pdv_data***)
-	vic_bit_map	24	192b	See explanation in 3.1 supra
36	vic_data	2 + 1..999	4i + 1..1998x	The first two bytes announce the length of the field (only digits), **these two bytes not included**. Contains APDU data to be given to the eID application directly.
171	vic_bit_map_application_id	4	128b	EIC application identifier: **00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 00**
179	vic_version	1	2i	Must always be **13** (vic_protocol_subversion 1.07/13)

Note:
Due to the multiple message exchanges to read the information on the card, it can take a while to collect information from the eID with the standard communication parameters; especially for the transmission of the picture. Therefore, some improvements have been proposed to accelerate the card reading process (higher baudrate, high level APDU

commands, etc.). These improvements enable the eID application to collect all information from the card in less than 7 seconds.
Please contact our Customer Support service if you are interested in getting more details about this.

5.9 Belgian SIS card reading (Social Identity System)

The Belgian SIS card reading is an application limited to the Belgian market. It requires the presence of the corresponding application on the terminal, which is not present by default.

This application works in the same way than the *Belgian ID card reading* application (see above):

- the exchange of ***vmc_purse_request*** and ***pdv_purse_result*** messages enables to identify the SIS BPA application in the `vic_bit_map_application_id` of the response:

...	...	...	...	...
171	vic_bit_map_application_id	16	128b	SIS BPA Application identifier: **00 00 00 00 00 00 00 00 00 00 00 00 00 20 00 00**
...	...	...	...	...

- BPA protocol commands are encapsulated in the `vic_data` field of exchanges ***vmc_data***/***pdv_data*** messages

Details on the BPA protocol is out of scope of this document. Please refer to appropriate documentation.

CONFIDENTIALITY

The information in this document is confidential and shall not be disclosed to any third party in whole or in part without the prior written consent of Atos Worldline S.A./N.V. The Requester shall duly remit a copy of the signed non-disclosure that is attached in annex.

COPYRIGHT

The information in this document is subject to change without notice and shall not be construed as a commitment by Atos Worldline S.A./N.V.
The content of this document, including but not limited to trademarks, designs, logos, text, images, is the property of Atos Worldline S.A/N.V. and is protected by the Belgian Act of 30.06.1994 related to author's right and by the other applicable Acts.
The contents of this document must not be reproduced in any form whatsoever, by or on behalf of third parties, without the prior written consent of Atos Worldline S.A./N.V.
Except with respect to the limited license to download and print certain material from this document for non-commercial and personal use only, nothing contained in this document shall grant any license or right to use any of Atos Worldline S.A./N.V. 's proprietary material.

LEGAL DISCLAIMER

While Atos Worldline S.A./N.V. has made every attempt to ensure that the information contained in this document is correct, Atos Worldline S.A.:N.V. does not provide any legal of commercial warranty on the document that is described in this specification. The technology is thus provided "as is" without warranties of any kind, expressed or implied, included those of merchantability and fitness for a particular purpose. Atos Worldline S.A./N.V. does not warrant or assume any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, product or process disclosed.
To the fullest extend permitted under applicable law, neither Atos Worldline S.A./N.V. nor its affiliates, directors, employees and agents shall be liable to any party for any damages that might result from the use of the technology as described in this document (including without limitation direct, indirect, incidental, special, consequential and punitive damages, lost profits).

JURISDICTION AND APPLICABLE LAW

These terms shall be governed by and construed in accordance with the laws of Belgium.
You irrevocably consent to the jurisdiction of the courts located in Brussels for any action arising from or related to the use of this document.

© ATOS WORLDLINE, 2012